

Entrega Final de Proyecto

Consignas de Entrega de Proyecto

Framework de Automatización de Pruebas

El objetivo del Trabajo Final Integrador es que desarrolles un framework de automatización de pruebas completo, que combine todos los conocimientos adquiridos a lo largo del curso. Este proyecto consistirá en la creación de un framework de testing automatizado en Python, utilizando Selenium WebDriver para pruebas de UI, la biblioteca Requests para pruebas de API, y aplicando patrones de diseño como Page Object Model para estructurar el código de manera eficiente y mantenible. Tu proyecto deberá demostrar una sólida comprensión de las técnicas de automatización, incluir pruebas tanto de UI como



Modelo de entrega:

[Haz clic aquí](#)

de API, y generar reportes visuales que faciliten la interpretación de los resultados.

Requerimientos Específicos:

Tecnologías a Utilizar:

1. Python como lenguaje de programación principal
2. Pytest como framework de testing
3. Selenium WebDriver para automatización de interfaces web
4. Biblioteca Requests para pruebas de API
5. Git para control de versiones
6. GitHub como repositorio de código



Organización del Código:

- Estructura tu proyecto con una organización clara de directorios (pages, tests, utils, etc.)
- Implementa el patrón Page Object Model para las pruebas de UI
- Aplica buenas prácticas de programación y comentarios descriptivos
- Nombra variables, métodos y clases de manera significativa

1: Funcionalidades Específicas:

Obligatorio

1. Pruebas de UI (Selenium WebDriver):

Automatización de Casos de Prueba:

- Implementa al menos 5 casos de prueba para un sitio web demo (puede ser saucedemo.com, automationpractice.com, o cualquier otro sitio de práctica)
- Los casos de prueba deben cubrir flujos completos (ejemplo: login, navegación, búsqueda, añadir producto al carrito, checkout)
- Incluye al menos un escenario negativo (ejemplo: login con credenciales inválidas)

Manejo de Datos de Prueba:

- Implementa alguna forma de parametrización para ejecutar las pruebas con diferentes conjuntos de datos
- Utiliza fuentes externas (archivos CSV, JSON, etc.) para leer los datos de prueba

1: Funcionalidades Específicas:

Obligatorio

- **Implementación de Page Object Model:**

- Crea clases para cada página que vayas a automatizar
- Implementa métodos que representen acciones en cada página
- Separa la lógica de las pruebas de la lógica de interacción con la página

- **Gestión de Capturas de Pantalla:**

- Implementa una funcionalidad que capture screenshots automáticamente cuando una prueba falla
- Almacena las capturas con nombres descriptivos que incluyan fecha/hora y nombre del test

2. Pruebas de API (Requests):

Obligatorio

- **Automatización de Endpoints:**

- Implementa al menos 3 casos de prueba para una API pública (puede ser ReqRes, JSONPlaceholder, o cualquier otra API pública)
- Cubre diferentes métodos HTTP (GET, POST, DELETE)

- **Validación de Respuestas:**

- Verifica códigos de estado HTTP
- Valida estructura y contenido de las respuestas JSON

- **Encadenamiento de Peticiones (Opcional):**

- Implementa un flujo donde una petición dependa del resultado de otra (ejemplo: crear un recurso y luego obtenerlo)

- Implementa assertions para diferentes escenarios (éxito, error, etc.)

3. Generación de Reportes:

Obligatorio

- **Reportes HTML:**

- Configura pytest para generar reportes HTML detallados
- Los reportes deben mostrar claramente los tests ejecutados, su estado (pasado/fallado) y duración
- Incluye capturas de pantalla en los reportes para las pruebas fallidas

- **Logging:**

- Implementa un sistema de logging que registre pasos clave durante la ejecución
- El log debe ser lo suficientemente detallado para facilitar la depuración

4. Integración con CI/CD:

Opcional



GitHub Actions

Github Actions

- Configura GitHub Actions para ejecutar tus pruebas automatizadas cuando se realice un push al repositorio
- Genera y almacena los reportes como artefactos de la ejecución

5. Control de Versiones y Documentación:

Obligatorio

Repositorio en GitHub:

- Sube el proyecto a un repositorio en GitHub

README.md:

- Incluye un archivo README.md que explique:

- Mantén un historial de commits que documente el progreso del proyecto
 - Usa ramas para desarrollar funcionalidades y luego fusiónalas con la rama principal
- El propósito del proyecto
 - Las tecnologías utilizadas
 - La estructura del proyecto
 - ¿Cómo instalar las dependencias?
 - ¿Cómo ejecutar las pruebas?
 - ¿Cómo interpretar los reportes generados?

Funcionalidad Esperada:

- El framework debe ser capaz de ejecutar todas las pruebas de manera consistente
- Las pruebas deben ser independientes entre sí (la falla de una no debe afectar a las demás)
- Los reportes generados deben proporcionar información clara sobre los resultados
- La estructura del código debe facilitar la incorporación de nuevas pruebas

Formato de Entrega:

- El proyecto debe estar subido a un repositorio público en GitHub

Nombre del Repositorio:

proyecto-final-automation-testing-[nombre-apellido]

- Comparte el enlace del repositorio antes de la fecha límite